Innovation partner H2S HACK2SKILL

Media partner YOURSTORY

AI for Bharat Hackathon

Powered by aws

Team Name : PleaseOfferMeAJob

Team Leader Name : Shashank Dubey

Problem Statement : India’s healthcare system relies on unstructured, multilingual paper records, leading to fragmented patient histories, delayed insurance claims, and increased fraud.

Brief about the Idea:

AI-Powered Longitudinal Health Intelligence for Bharat

OpenClaims transforms unstructured medical documents into a structured, evolving health graph for every patient.

In India, healthcare records are fragmented across hospitals, insurance providers, and paper documents. Patients carry physical files, medical history gets lost, and every consultation often starts from scratch.

OpenClaims solves this by:

- Extracting information from medical documents using AI
- Structuring it into meaningful medical entities
- Building a longitudinal graph-based timeline
- Enabling better clinical insight, analytics, and future risk modeling

This creates a unified, intelligent health memory for every patient.

Why AI is Required in This Solution

Healthcare documents are:

- Unstructured (PDFs, scanned copies)
- Inconsistent in format
- Written in natural language
- Contain complex medical terminology

Traditional rule-based systems cannot reliably extract structured medical intelligence.

AI enables:

- Intelligent text extraction from documents
- Entity recognition (diagnosis, claims, encounters)
- Structured transformation into graph-ready data
- Multi-hop relationship understanding

AI converts static documents into connected medical knowledge.

How AWS Services Are Used

Our architecture is fully cloud-native and deployed on AWS:

- Amazon S3 – Secure storage for uploaded medical documents
- AWS Textract – Extracts text from scanned and digital PDFs
- Amazon Bedrock – LLM-powered structured data extraction
- Amazon EC2 – Hosts containerized backend services
- Amazon RDS (Postgres) – Stores metadata and processing status
- Redis (Queue Layer) – Handles asynchronous document processing
- Graph Database Layer – Builds longitudinal patient graph

This architecture ensures scalability, reliability, and modularity.

Value the AI Layer Adds

The AI layer enables:

- Conversion of unstructured medical text into structured entities
- Longitudinal health timeline construction
- Faster clinical context understanding
- Foundation for preventive risk intelligence
- Future natural-language graph querying

Without AI, the system would only store documents.
With AI, the system understands relationships.

List of features offered by the solution

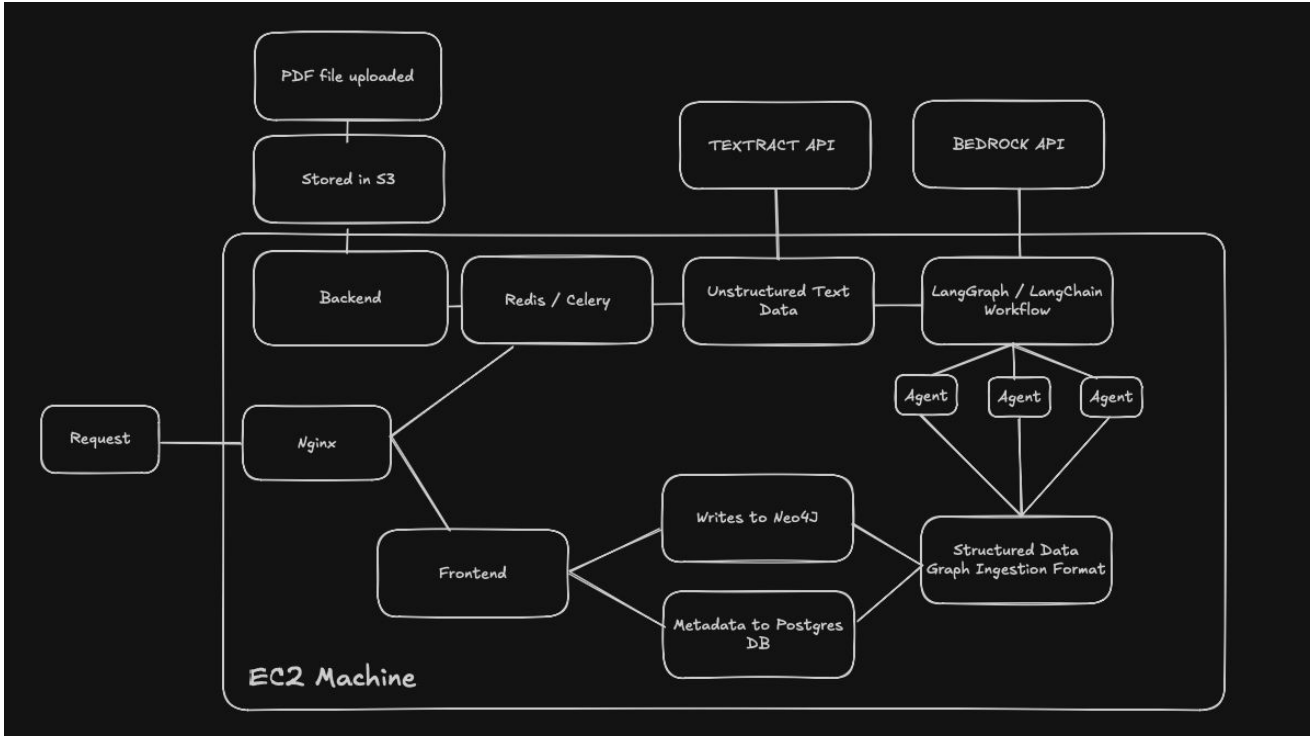
Core Features

- Patient profile creation
- Secure medical document upload
- Automated document text extraction
- AI-powered structured medical parsing
- Graph-based longitudinal health timeline
- Duplicate-safe longitudinal append logic
- Hospital and claim relationship mapping
- Basic graph querying (encounters, diagnoses, claims)

Future-Ready Capabilities

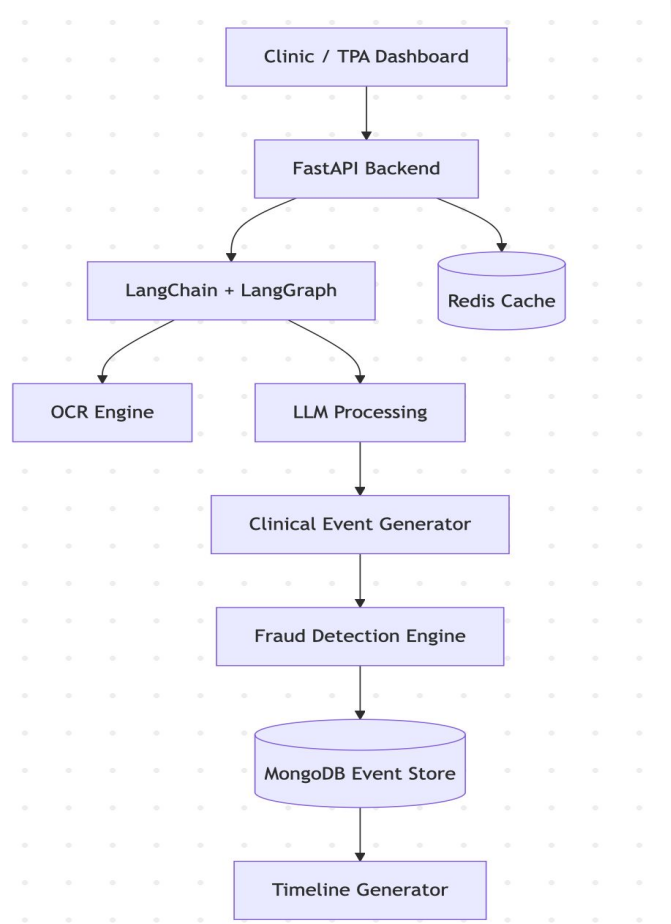
- Natural language querying over patient graph
- Preventive risk score modeling
- Cross-patient analytics
- Insurance intelligence layer

Process flow diagram or Use-case diagram

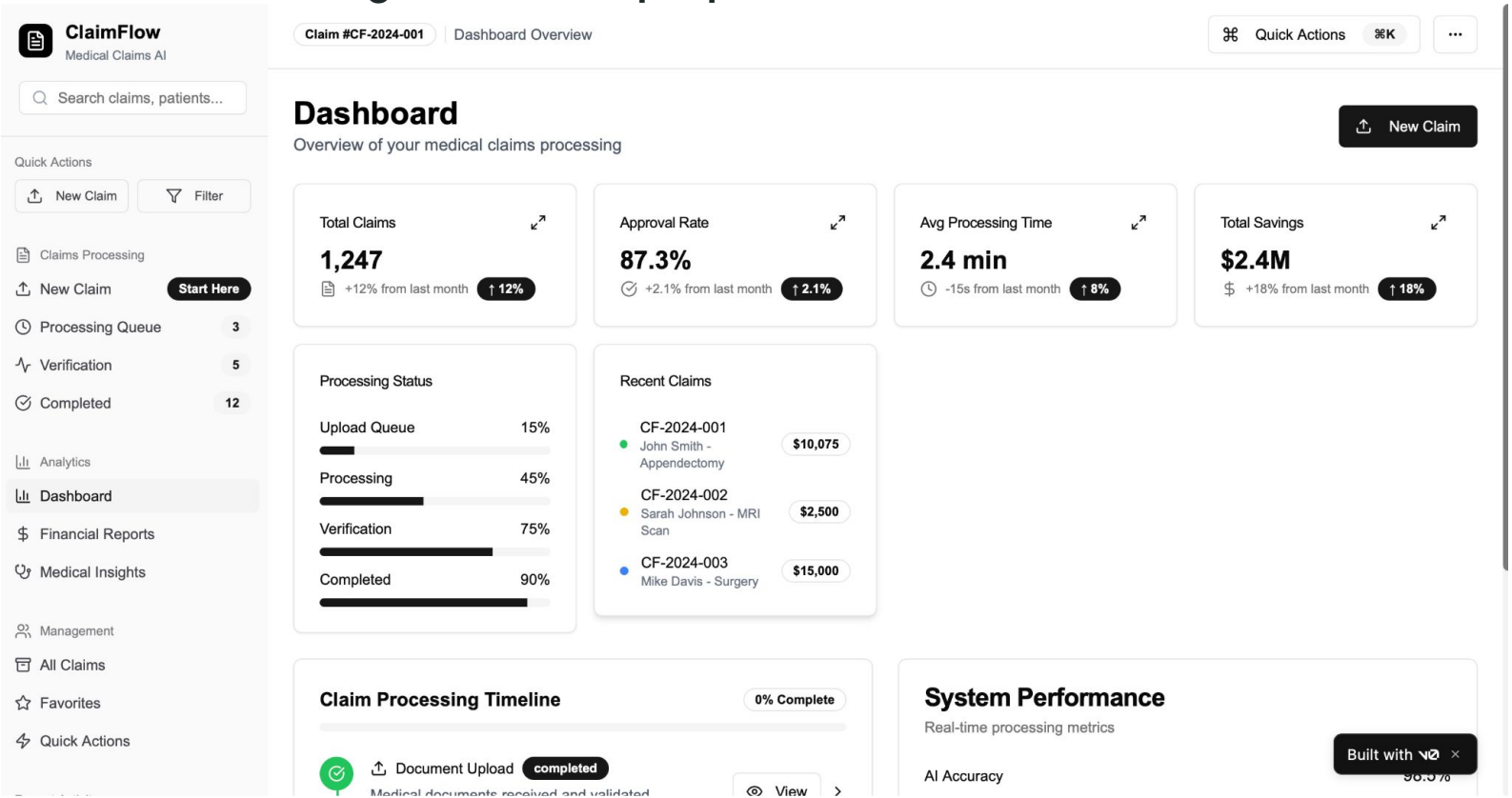


High-Level Architecture Overview

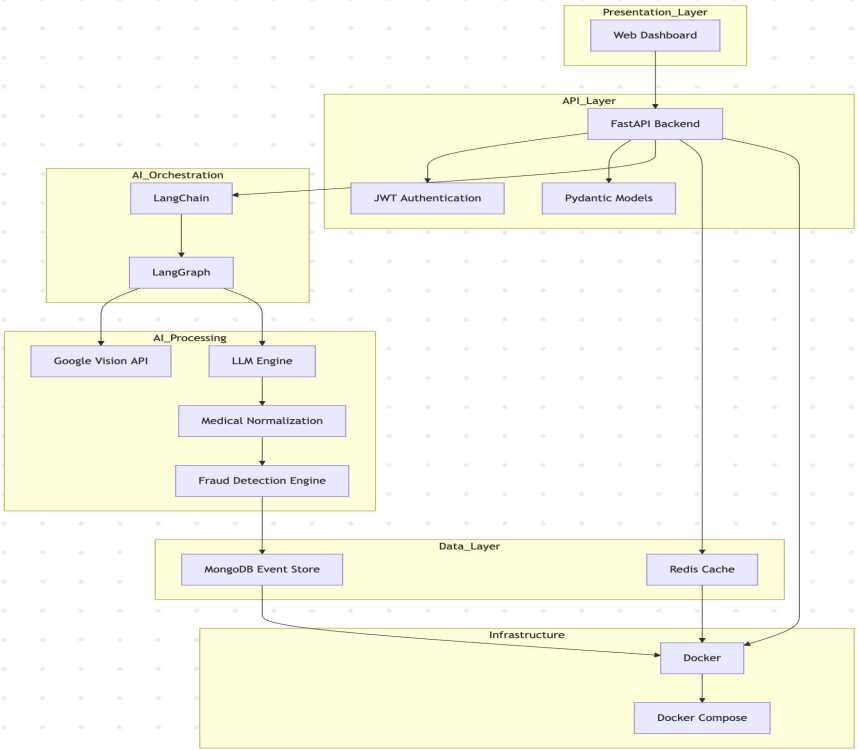
- Frontend Dashboard (Clinic / TPA Interface)
- FastAPI Backend (API Layer)
- LangChain + LangGraph (AI Orchestration)
- OCR Engine (Google Vision – Replaceable with AWS Textrac
- LLM Engine (OpenAI / Bedrock / Local LLM)
- MongoDB (Event Store)
- Redis (Caching / Task Handling)
- Docker Compose Deployment



Wireframes/Mock diagrams of the proposed solution



Architecture diagram of the proposed solution:



Technologies utilized in the solution:

Backend & Processing

- Python 3.11
- FastAPI
- Celery (asynchronous processing)
- Redis (queue broker)
- SQLAlchemy
- Pydantic

AI & Data Processing

- AWS Textract
- Amazon Bedrock (LLM inference)
- LangGraph
- LangChain

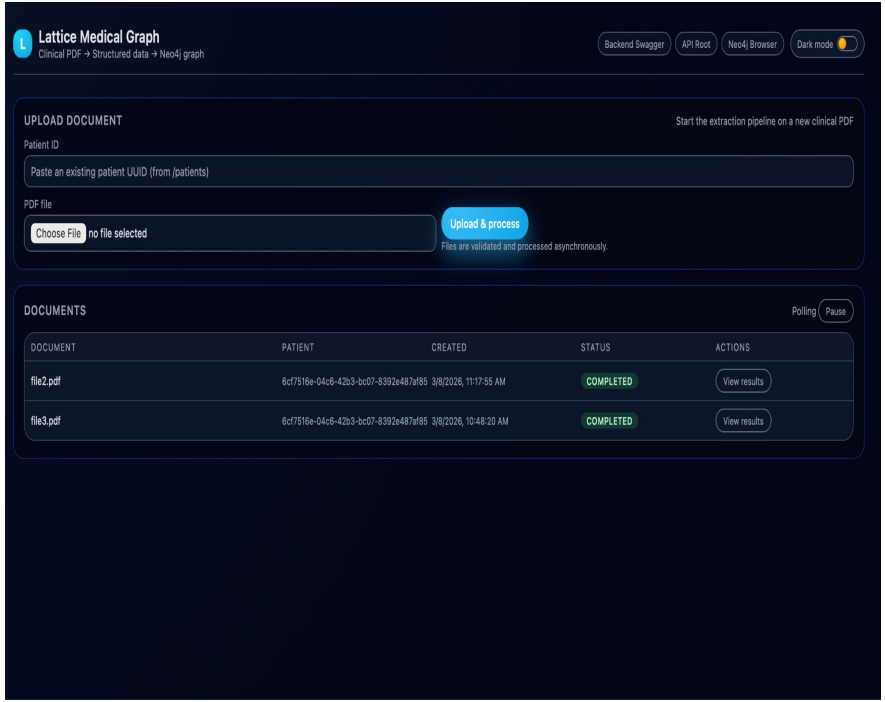
Databases

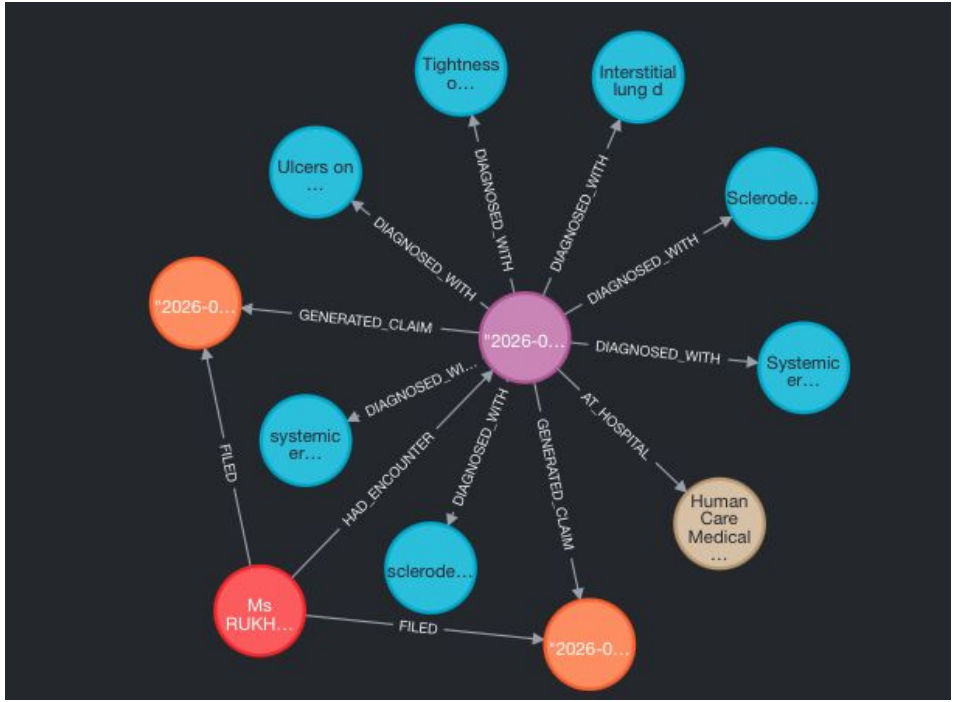
- PostgreSQL (metadata)
- Neo4j (Graph Database – demo environment)
- Neptune-compatible architecture (production-ready design)

Infrastructure

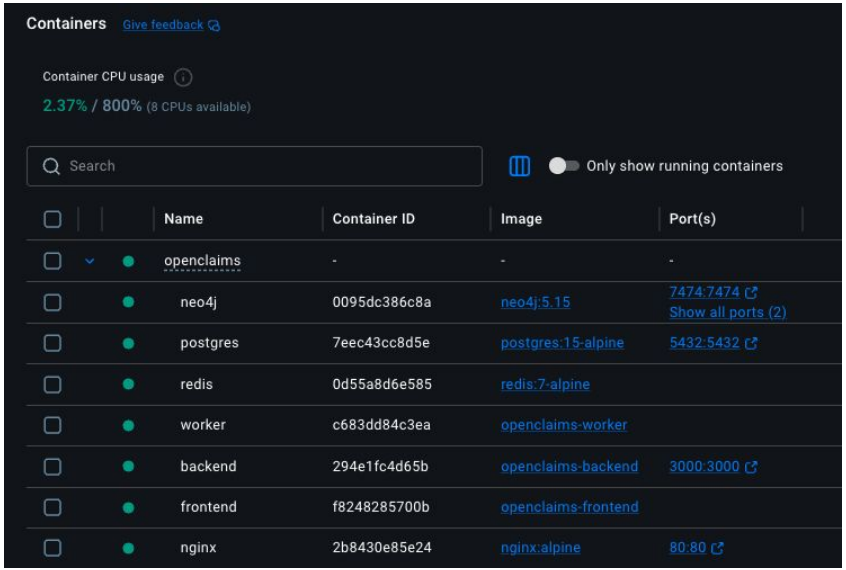
- Docker & Docker Compose
- Nginx Reverse Proxy
- AWS EC2
- AWS S3
- AWS RDS

Snapshots of the prototype:

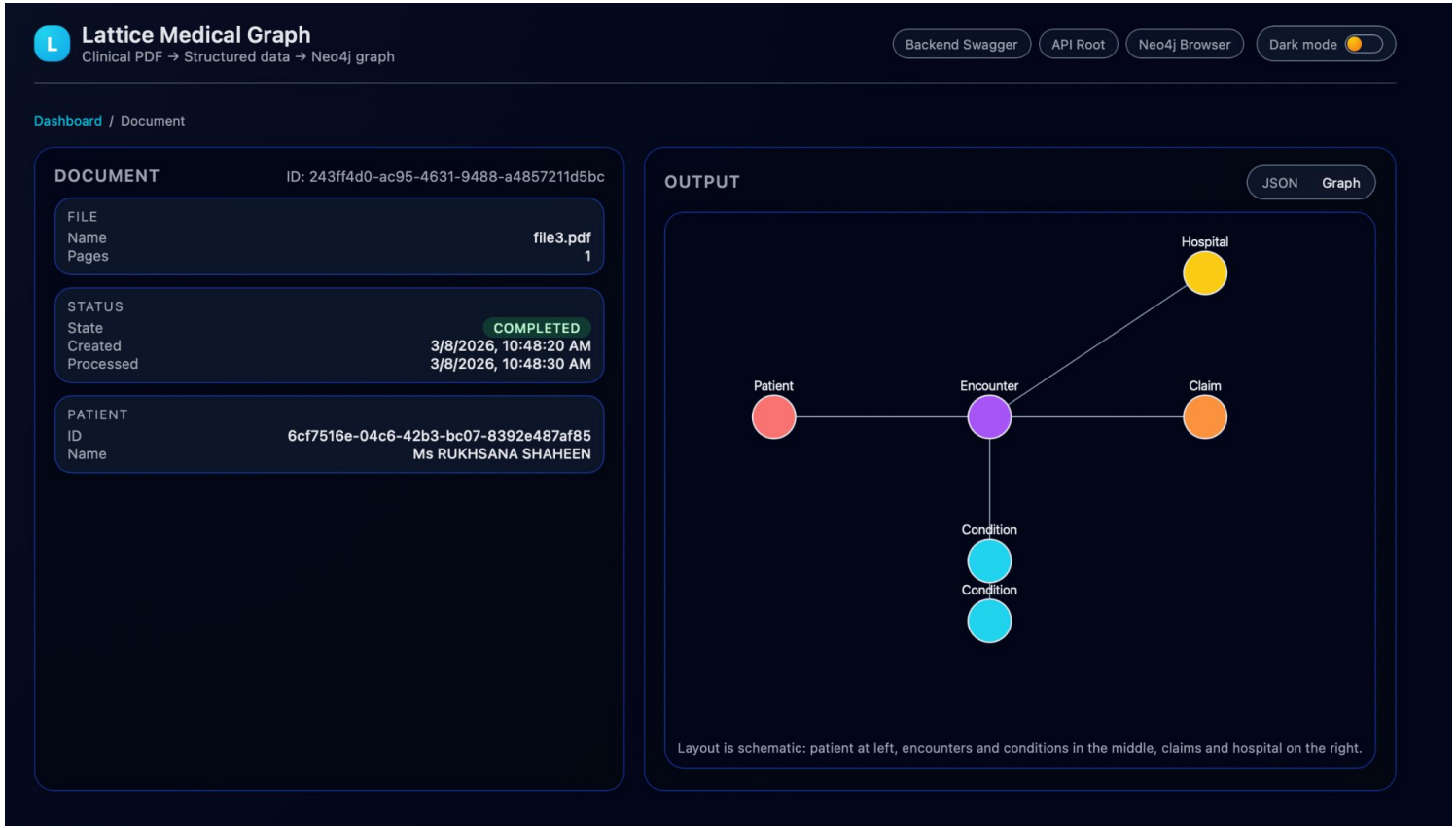




Snapshots of the prototype:



Snapshots of the prototype:



Prototype Performance report/Benchmarking:

Document Processing Performance

- Average document processing time: ~5–15 seconds (Textract + LLM + Graph ingestion)
- Asynchronous queue processing ensures:
  - Non-blocking user uploads
  - Scalable document handling
  - Clean status tracking

System Reliability

- Idempotent graph writes (MERGE-based logic)
- No duplicate patient nodes
- Consistent longitudinal append behavior
- Safe, read-only graph queries

Scalability Potential

- Horizontally scalable worker architecture
- Cloud-native deployment
- Modular AI pipeline
- Graph DB optimized for multi-hop queries

The system is designed for production-grade extensibility.

Additional Details/Future Development (if any):

Planned Enhancements:

1. **Natural Language Graph Querying**
   Example:
   "Show diabetes history over the last 2 years."

2. **Preventive Risk Intelligence**
   - Identify repeated cardiac complaints
   - Early detection patterns
   - Proactive health recommendations

3. **Cross-Hospital Interoperability**
   - Unified patient graph across institutions
   - Reduced duplicate testing

4. **Advanced Insurance Intelligence**
   - Fraud detection patterns
   - Transparent claim analytics
   - Preventive insurance models

Prototype Assets:

- ➔ GitHub Public Repository Link : https://github.com/CorruptEntity0982/Lattice
- ➔ Demo Video Link : https://youtu.be/8dR3FNTtLI4?si=PpzTaF70H9TIXu0E
- ➔

Innovation partner H2S HACK2SKILL | Media partner YOURSTORY

AI for Bharat Hackathon

Powered by aws

Thank You